

Arquitectura Serverless

Diagrama de Arquitectura:

La arquitectura diseñada para este proyecto en Google Cloud Platform (GCP) se compone de una VPC única que integra cuatro subredes especializadas, garantizando escalabilidad, resiliencia y alto rendimiento. La primera subred (10.109.2.0/27) aloja la instancia *worker*, encargada de ejecutar microservicios clave para el procesamiento de datos, incluyendo la transformación de documentos, la inserción en una base de datos vectorial (PostgreSQL) mediante Cloud SQL (PaaS) y la comunicación con el modelo Gemini para el análisis avanzado de consultas. La segunda subred (10.108.0.0/26) despliega un grupo de autoescalado con el microservicio de *frontend*, que gestiona la interfaz de usuario, junto con un microservicio en Cloud Run que maneja solicitudes de autenticación, registro, almacenamiento de documentos en Cloud Storage y la publicación de mensajes en Pub/Sub para su procesamiento por parte de los *workers*.

La tercera subred (10.0.0.0/26) actúa como capa de proxy, albergando un balanceador de carga que distribuye el tráfico externo hacia las instancias saludables del grupo de autoescalado *frontend*, mejorando la confiabilidad del sistema. Por su parte, la cuarta subred (10.109.0.0/26) ejecuta en Cloud Run el microservicio de *chunking*, el cual procesa los mensajes de Pub/Sub para fragmentar el texto antes de su análisis.

La arquitectura se complementa con servicios gestionados de GCP, como un balanceador de carga interno para distribuir tráfico desde el servicio de *chunking* hacia los *workers*. Donde se hace uso de Artifact Registry para centralizar imágenes Docker, y Gemini para capacidades avanzadas de NLP. Además, Cloud SQL soporta bases de datos relacionales y vectoriales, mientras que Cloud Storage proporciona almacenamiento seguro para documentos. Esta estructura integrada asegura un flujo eficiente de datos, desde la interacción del usuario hasta el procesamiento y almacenamiento final.

Justificación de componentes reemplazados:

En este proyecto, se priorizó la optimización del rendimiento y la escalabilidad mediante el reemplazo de dos componentes críticos. En primer lugar, se separó el microservicio del *frontend* y el *backend*, los cuales inicialmente coexistían en un mismo grupo de instancias con escalamiento lento. Para solucionar esto, se migró el *backend* a **Cloud Run**, un servicio que permite una escalabilidad autogestionada por GCP más rápida que la configuración manual y evita que las cargas elevadas afecten el *frontend*, garantizando así una mejor distribución de recursos y una respuesta ágil ante picos de demanda.

El segundo cambio clave fue la migración del microservicio de *chunking*, que originalmente se ejecutaba en el mismo grupo de instancias destinado a los *workers*. Al trasladarlo a **Cloud Run**, se redujo la carga en las instancias críticas, dejando únicamente tres microservicios esenciales en el grupo de *workers* para un procesamiento más eficiente. Esta modificación no solo mejoró la escalabilidad del servicio de *chunking*,

sino que también aseguró que las tareas principales de procesamiento no se vieran afectadas por fluctuaciones en la demanda. Ambos ajustes contribuyen a una arquitectura más robusta, flexible y preparada para manejar cargas variables sin comprometer el rendimiento del sistema.

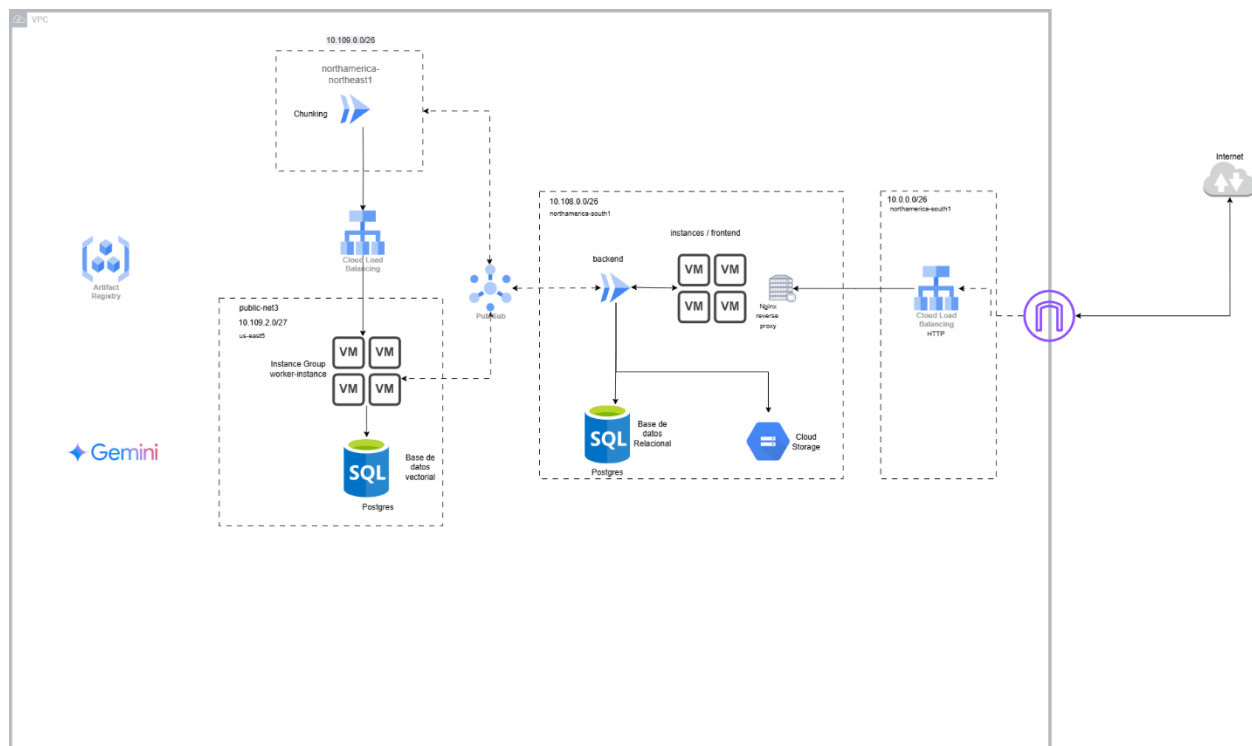


Figura 1: Diagrama de Arquitectura.

Pasos de integración:

El proceso de integración inicia con el desacoplamiento de los microservicios de *backend* y *chunking*, migrándolos a Cloud Run para mejorar su escalabilidad. El *backend* se despliega en la misma subred que aloja el *frontend*, manteniendo una comunicación directa y eficiente entre ambos, mientras que su conexión con la base de datos relacional, Pub/Sub y Cloud Storage no requiere configuraciones adicionales, ya que estos servicios conservan sus mismos *endpoints*. Por otro lado, el microservicio de *chunking*, alojado en una subred distinta y en otra región, se integra mediante un balanceador de carga global, que distribuye las tareas procesadas hacia el grupo de instancias de *workers* en la subred principal, asegurando un reparto equitativo de la carga. Los *workers* mantienen su comunicación con Pub/Sub, la base de datos vectorial (a través de Cloud SQL) y los servicios de Gemini, sin modificaciones en sus *endpoints*, lo que garantiza continuidad en el flujo de procesamiento y minimiza impactos en la arquitectura existente. Esta estrategia de integración optimiza el rendimiento, la escalabilidad y la resiliencia del sistema sin introducir complejidades innecesarias.

Variables de Entorno:

Se configuraron dos variables clave para garantizar la seguridad y flexibilidad del sistema:

1. ***DATABASE_URL***: Almacena el string de conexión a la base de datos, evitando que credenciales sensibles queden expuestas en el build de la imagen desplegada en Cloud Run. Esto protege la información de los usuarios y facilita la rotación de credenciales sin modificar el código.
2. ***CLOUD_STORAGE***: Contiene el nombre del bucket de Cloud Storage donde se guardan los documentos subidos por los usuarios, evitando referencias directas en el código y reduciendo riesgos de exposición accidental de datos confidenciales.

Ambas variables siguen las mejores prácticas de seguridad, manteniendo la configuración fuera del código fuente y permitiendo una gestión centralizada a través de los servicios gestionados de GCP, lo que mejora tanto la protección de datos como la mantenibilidad del sistema.

Costos aproximados:

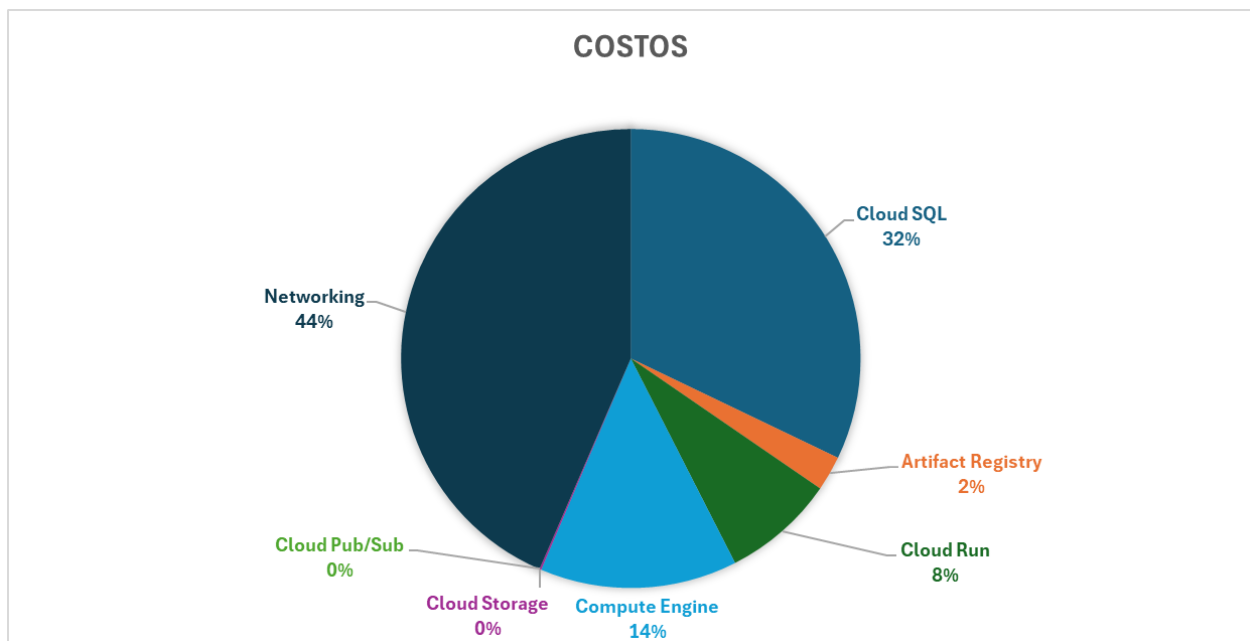


Figura 2. Distribución de costos por servicios.

Descripción del Servicio	Costo (COP)
Cloud SQL	38,589.26
Artifact Registry	2,955.53
Cloud Run	9,507.16
Compute Engine	16,680.56
Cloud Storage	149.88
Cloud Pub/Sub	0.05
Networking	52,244.6

Tabla 1. Costo en COP de los servicios usados.

De acuerdo con los gráficos de distribución de consumo y la tabla de costos (Figura 2 y Tabla 1), el costo total estimado del proyecto asciende a 120.127,04 COP, siendo los servicios de Networking y Cloud SQL los que representan la mayor parte del gasto. Este incremento en los costos de red probablemente se deba al tráfico interregional, así como a las transferencias de datos desde y hacia Internet. En cuanto a Cloud SQL, el alto consumo puede atribuirse a las operaciones constantes de lectura y escritura en las bases de datos alojadas (tanto la base de datos vectorial como la de usuarios), las cuales requieren recursos computacionales y de almacenamiento significativos. Estos resultados resaltan la importancia de optimizar el uso de estos servicios, considerando estrategias como el caching de consultas frecuentes o la revisión de las configuraciones de red para reducir costos sin comprometer el rendimiento del sistema.

Limitaciones:

La principal limitación encontrada estuvo relacionada con las restricciones de la capa gratuita de GCP, que impuso un tope en la cantidad de recursos desplegados en una sola región. Esta situación nos obligó a distribuir la infraestructura en múltiples regiones, lo que generó dos desafíos clave: (1) la necesidad de implementar configuraciones adicionales para garantizar la comunicación entre servicios distribuidos geográficamente, y (2) un potencial incremento en la latencia del tráfico interregional. Estas complicaciones técnicas surgieron como consecuencia directa de trabajar dentro de los límites del tier gratuito, demostrando cómo las restricciones de recursos pueden impactar tanto en la arquitectura como en el desempeño del sistema.